

MODUL 6

(MORE ON SORTING)

DESKRIPSI TEMA

Quick Sort adalah algoritma pengurutan yang menggunakan metode *divide and conquer*. Algoritma ini sering di implementasikan untuk data dengan jumlah yang banyak. Konsep utama algoritma ini adalah menentukan sebuah titik (*pivot*) kemudian membagi elemen dalam list, menjadi dua partisi yaitu: partisi yang lebih kecil dari *pivot* dan partisi yang lebih besar dari *pivot*. Kemudian tiap partisi akan di quick sort (rekursif) hingga elemen sudah terurut.

Merge Sort adalah algoritma pengurutan yang menggunakan metode *divide and conquer*. Algoritma ini membagi sebuah array menjadi 2 bagian, dan tiap bagian tersebut akan terus dibagi 2 hingga akhirnya hanya terdiri dari 1 elemen saja.

Shell Sort merupakan variasi dari Insertion Sort. Dalam Insertion Sort, kita hanya memindahkan element ke 1 posisi ke depan, ketika sebuah elemen harus dipindahkan jauh ke depan, maka akan ada banyak pemindahan yang dilakukan. Sedangkan Shellsort, dapat menukarkan suatu elemen dengan elemen lainnya yang jauh.

Sebelum mempelajari tentang heap sort, ada beberapa terminologi dalam heap yang perlu dimengerti terlebih dahulu, yaitu *Heap Tree*, *Heapify*, dan *Heap Sort* itu sendiri.

Heap Tree merupakan complete binary tree yang dapat dikategorikan menjadi 2 berdasarkan pengurutan nilainya.

- Max Heap Tree, yaitu heap tree yang nilai parent-nya selalu lebih besar dari nilai child-nya.
- Min Heap Tree, yaitu heap tree yang nilai parent-nya selalu lebih kecil dari nilai child-nya.

Heapify secara sederhana merupakan proses untuk menciptakan struktur *heap* yang benar. Heap tree dapat diimplementasikan untuk melakukan proses pengurutan data, proses ini biasa disebut dengan *heap sort*. Dalam proses heap sort dibutuhkan struktur heap yang benar. Proses inilah yang disebut dengan *heapify*.

Heap sort merupakan sebuah proses pengurutan data dengan memanfaatkan struktur heap.

CAPAIAN PEMBELAJARAN MINGGUAN (SUB-CAPAIAN PEMBELAJARAN)

Mahasiswa dapat memahami dan menerapkan berbagai algoritma sorting lanjutan (C3).

PENUNJANG PRAKTIKUM

1. Software Codeblocks

LANGKAH-LANGKAH PRAKTIKUM

Tutorial 1 – Quick Sort

1. Simulasi

partition(bil, l, r); // l = 0, r = 8

i	0	1	2	3	4	5	6	7	8	
	2	3	8	7	6	1	4	9	5	pivot = r = 5
i	2	3	8	7	6	1	4	9	5	pivot = r = 5
i	2	3	8	7	6	1	4	9	5	pivot = r = 5
i	2	3	8	7	6	1	4	9	5	pivot = r = 5
i	2	3	8	7	6	1	4	9	5	pivot = r = 5
i	2	3	8	7	6	1	4	9	5	pivot = r = 5
i	2	3	8	7	6	1	4	9	5	pivot = r = 5
i	2	3	8	7	6	1	4	9	5	pivot = r = 5
i	2	3	1	7	6	8	4	9	5	pivot = r = 5
i	2	3	1	4	6	8	7	9	5	pivot = r = 5
i	2	3	1	4	6	8	7	9	5	pivot = r = 5
i	2	3	1	4	5	8	7	9	6	pivot = r = 5

2. Implementasi

```

1  #include <stdio.h>
2  #include <malloc.h>
3
4  /*  SWAP
5     Fungsi untuk menukar isi dua elemen */
6  void swap(int *a, int *b)
7  {
8      int temp;
9      temp = *a;
10     *a = *b;
11     *b = temp;
12 }

```

```

14 /*  PARTITIONING
15     Elemen terakhir dalam list dijadikan pivot.
16     Elemen dengan nilai lebih kecil dari pivot diswap ke kiri pivot.
17     Elemen dengan nilai lebih besar dari pivot diswap ke kanan pivot. */
18 int partition(int *bil, int l, int r)
19 {
20     // Menjadikan elemen terakhir list sebagai pivot
21     int pivot = bil[r];
22     // Tembok yang menjadi pemisah partisi kiri dan kanan
23     // Pertama diletakkan di kiri elemen pertama
24     int i = l-1;
25
26     // Looping dari elemen paling kiri hingga elemen sebelum pivot
27     for (int j=l; j<=r-1; j++)
28     {
29         // Jika elemen lebih kecil sama dengan pivot
30         if (bil[j] <= pivot)
31         {
32             // Geser tembok
33             i++;
34             // Sekarang bil[i] elemen paling kanan dari partisi kiri
35             // bil[i] belum tentu lebih kecil dari pivot
36
37             // Tukar bil[i] dengan bil[j] yang sudah dicek
38             swap(&bil[i], &bil[j]);
39         }
40     }
41
42     // Tukar elemen paling kiri partisi kanan dengan pivot
43     swap(&bil[i + 1], &bil[r]);
44     // Mengembalikan index dari pivot yang sudah benar
45     return (i + 1);
46 }

```

```

48  /* QUICK SORT
49     Langkah tersebut diulang untuk setiap partisi.
50     (Partisi sebutan untuk seluruh elemen di kiri pivot atau
51     seluruh elemen di kanan pivot) */
52  void quickSort(int *bil, int l, int r)
53  {
54      if (l < r)
55      {
56          // pi adalah partitioning index
57          int pi = partition(bil, l, r);
58          // Setelah fungsi dijalankan
59          // bil[pi] (sebelumnya pivot) sudah berada pada posisi yang benar
60
61          // Secara terpisah dan rekursif
62          // melakukan quicksort untuk tiap partisi kiri dan kanan
63          quickSort(bil, l, pi - 1);
64          quickSort(bil, pi + 1, r);
65      }
66  }

```

```

68  int main(int argc, char const *argv[])
69  {
70      int *bil;
71      int n;
72      int i;
73
74      printf("Jumlah bilangan: ");
75      scanf("%d", &n);
76
77      bil = (int *) malloc(sizeof(int)*n);
78
79      for(i = 0; i < n; i++)
80      {
81          scanf("%d", &bil[i]);
82      }
83
84      quickSort(bil, 0, n-1);
85
86      printf("Hasil Pengurutan: ");
87      for(i = 0; i < n; i++)
88      {
89          printf("%d ", bil[i]);
90      }
91
92      return 0;
93  }

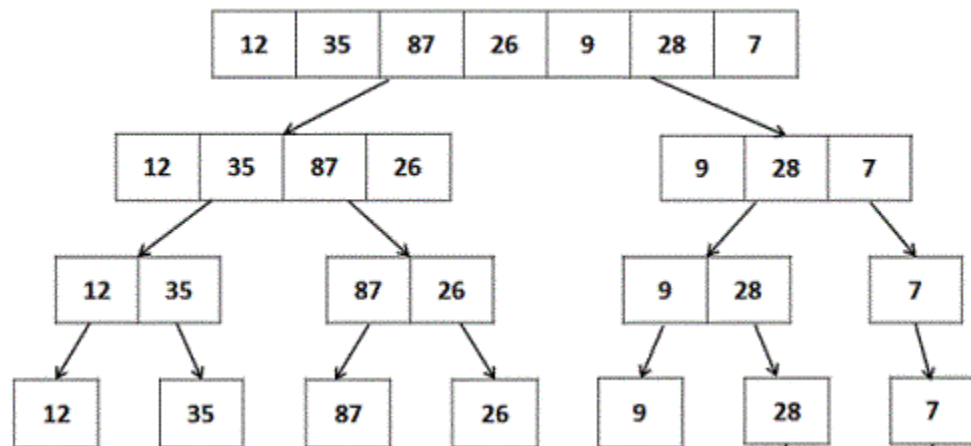
```

Tutorial 1.1

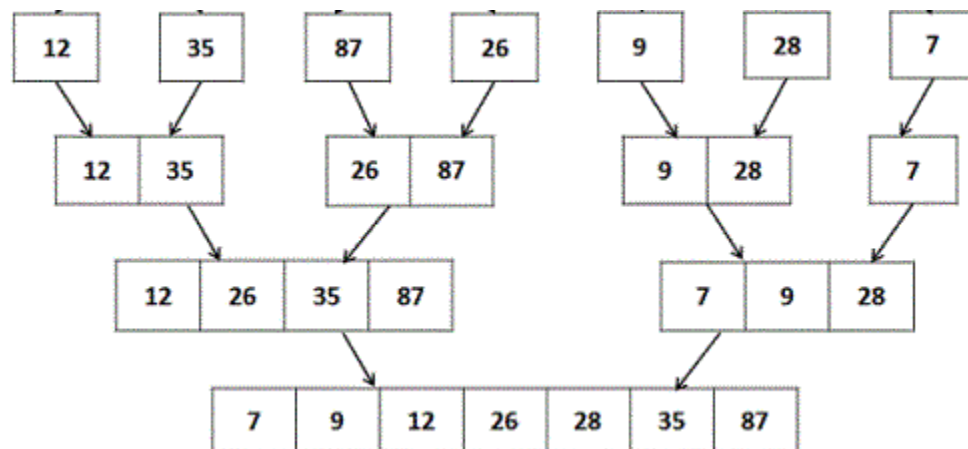
1. Ubahlah algoritma Quick Sort diatas sehingga nilai array menjadi terurut secara *descending*
2. Lalu simpan dengan nama file **Week07_Nama_NIM_1.c**

Tutorial 2 – Merge Sort

Merge Sort adalah algoritma pengurutan yang menggunakan metode *divide and conquer*. Algoritma ini membagi sebuah array menjadi 2 bagian, dan tiap bagian tersebut akan terus dibagi 2 hingga akhirnya hanya terdiri dari 1 elemen saja.



Sesudah itu, dari masing-masing elemen tersebut akan disambungkan kembali, sambil melakukan pengurutan.

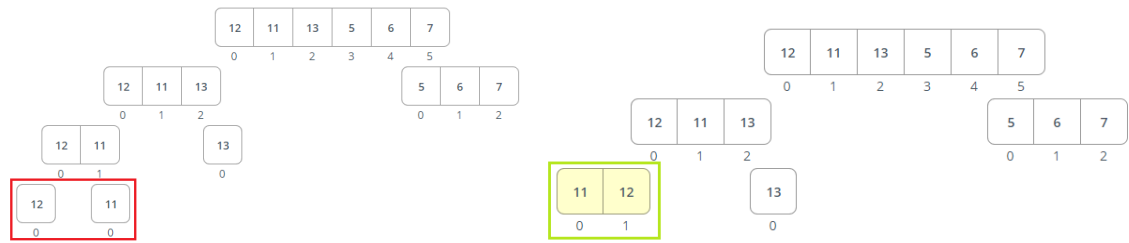


1. Simulasi

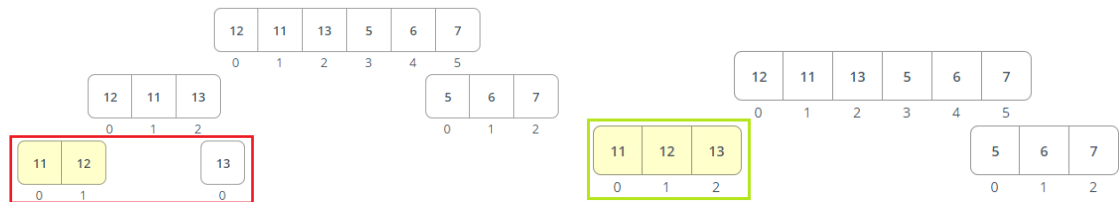
Visualisasi dari simulasi dapat dibuka pada tautan

<https://www.hackerearth.com/practice/algorithms/sorting/merge-sort/visualize/>

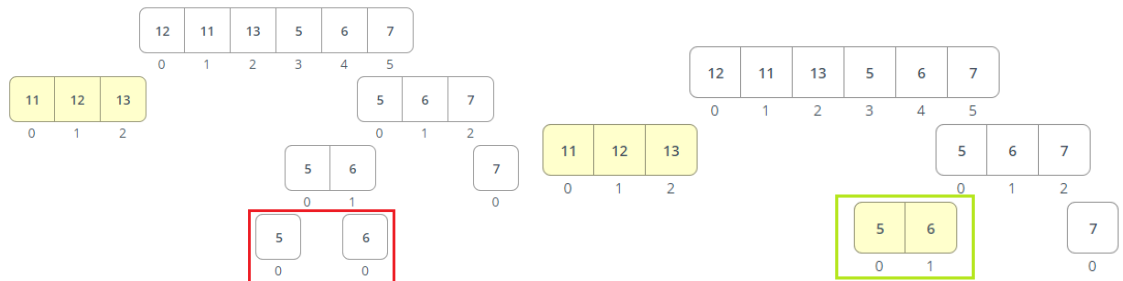
a. Tahap Pertama



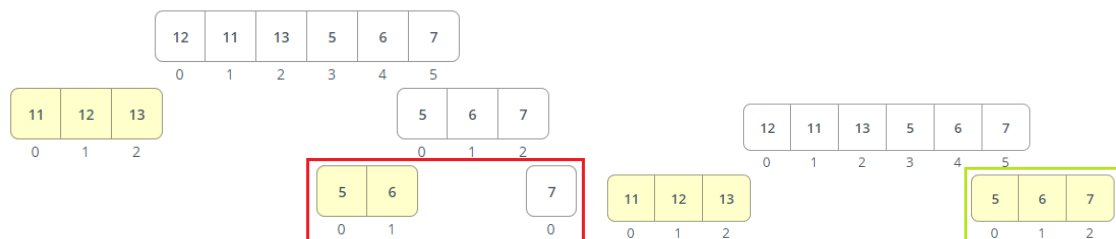
b. Tahap Kedua



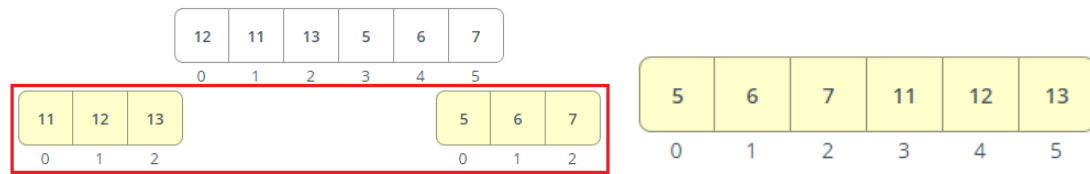
c. Tahap Ketiga



d. Tahap Keempat



e. Tahap Kelima



2. Implementasi

```
#include <stdio.h>

void merge(int *bil, int l, int m, int r){
    int i, j, k;

    //Banyak data di sisi kiri-----
    int n1 = m - l + 1;

    //Banyak data di sisi kanan-----
    int n2 = r - m;

    //Membuat array sementara untuk bilangan di sisi kiri-----
    int L[n1];

    //Membuat array sementara untuk bilangan di sisi kanan-----
    int R[n2];

    //Menyalin bilangan di sisi kiri ke array L[]-----
    for(i = 0; i < n1; i++){
        L[i] = bil[l + i];
    }

    //Menyalin bilangan di sisi kanan ke array R[]-----
    for(j = 0; j < n2; j++){
        R[j] = bil[m+1 + j];
    }

    /*Proses Menggabungkan kembali bilangan di sisi kiri dan kanan menjadi bil[l...r]*/
    i = 0; //Iterator untuk bilangan di sisi kiri-----
    j = 0; //Iterator untuk bilangan di sisi kanan-----
    k = l; //Iterator untuk menggabungkan kembali bilangan yang terpisah di sisi kanan dan sisi kiri---
```

```

/*Proses Menggabungkan kembali bilangan di sisi kiri dan kanan menjadi bil[l...r]*/
i = 0; //Iterator untuk bilangan di sisi kiri-----
j = 0; //Iterator untuk bilangan di sisi kanan-----
k = 1; //Iterator untuk menggabungkan kembali bilangan yang terpisah di sisi kanan dan sisi kiri----

while(i < n1 && j < n2){
    if(L[i] <= R[j]) {
        bil[k] = L[i];
        i++;
    }

    else{
        bil[k] = R[j];
        j++;
    }

    k++;
}

//Menyalin jika ada elemen yang tersisa dari L[]-----
while(i < n1){
    bil[k] = L[i];
    i++;
    k++;
}

//Menyalin jika ada elemen yang tersisa dari R[]-----
while(j < n2){
    bil[k] = R[j];
    j++;
    k++;
}
}

```

```

void mergeSort(int *bil, int l, int r){
    if(l < r){
        int m = (l+r)/2;

        mergeSort(bil, l, m);
        mergeSort(bil, m+1, r);

        //Proses ketika sudah menelusuri bagian kiri & kanan dari sebuah node-----
        merge(bil, l, m, r);
    }
}

```



```
int main(int argc, char const *argv[]){
    int *bil;
    int n;
    int i;

    printf("Jumlah bilangan: ");
    scanf("%d", &n);

    bil = (int *) malloc(sizeof(int)*n);

    for(i = 0; i < n; i++){
        scanf("%d", &bil[i]);
    }

    mergeSort(bil, 0, n-1);

    printf("Hasil Pengurutan: ");
    for(i = 0; i < n; i++){
        printf("%d ", bil[i]);
    }

    return 0;
}
```

Tutorial 2.1

1. Ubahlah algoritma Merge Sort di atas sehingga nilai array menjadi terurut secara *descending*
2. Lalu simpan dengan nama file **Week07_Nama_NIM_2.c**

Tutorial 3 – Shell Sort

1. Simulasi

Terdapat $n = 5$ bilangan, dengan nilai array = [12, 34, 54, 2, 3],

- Dimulai dari $gap = n/2 = 5/2 = 2$
- satu per satu pilih elemen j di sebelah kanan gap dan membandingkannya dengan sebelah kirinya dengan suatu jarak $[j - gap]$
- [12, 34, 54, 2, 3] $\rightarrow 12 > 54$? False
- [12, 34, 54, 2, 3] $\rightarrow 34 > 2$?
 - True, Swap [12, 2, 54, 34, 3]
- [12, 2, 54, 34, 3] $\rightarrow 54 > 3$?
 - True, Swap [12, 2, 3, 34, 54], 12 > 3 ?
 - True, Swap [3, 2, 12, 34, 54]
- Lalu, kurangi $gap = gap/2 = 2/2 = 1$

- satu per satu pilih elemen dari gap lagi dan bandingkan.
- [3, 2, 12, 34, 54], 3 > 2 ?
 - True, Swap [2, 3, 12, 34, 54]
- [2, 3, 12, 34, 54], 3 > 12 ? False
- [2, 3, 12, 34, 54], 12 > 34 ? False
- [2, 3, 12, 34, 54], 34 > 54 ? False
- Data telah terurutkan secara ascending.

2. Implementasi

```
#include <stdio.h>
#include <stdlib.h>

void shellSort(int *arr, int n)
{
    int i, j, gap;
    // mulai dari gap besar (n/2), lalu kurangi gapnya (gap /= 2)
    // NOTE : tidak harus menggunakan angka 2 sebagai pembagi
    // bisa gunakan angka yang lebih besar untuk perpindahan yang lebih jauh
    for(gap = n/2; gap > 0; gap /= 2)
    {
        // lakukan insertion sort untuk gap size ini.
        for(i = gap; i < n; i++)
        {
            // simpan arr[i] ke temp
            int temp = arr[i];

            // pindahkan nilai yang besar ke tempat yang seharusnya
            for(j = i; j >= gap && arr[j - gap] > temp; j -= gap)
                arr[j] = arr[j-gap];

            // taruh temp ke lokasi yang benar
            arr[j] = temp;
        }
    }
}
```

```

int main(int argc, char const *argv[])
{
    int i, n, *arr;
    printf("input N = ");
    scanf("%d", &n);
    arr = (int *) malloc(sizeof(int)*n);

    printf("Masukkan bilangan : ");
    for(i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    shellSort(arr, n);
    printf("Hasil Sort : ");
    for(i = 0; i < n; i++)
        printf("%d ", arr[i]);

    return 0;
}

```

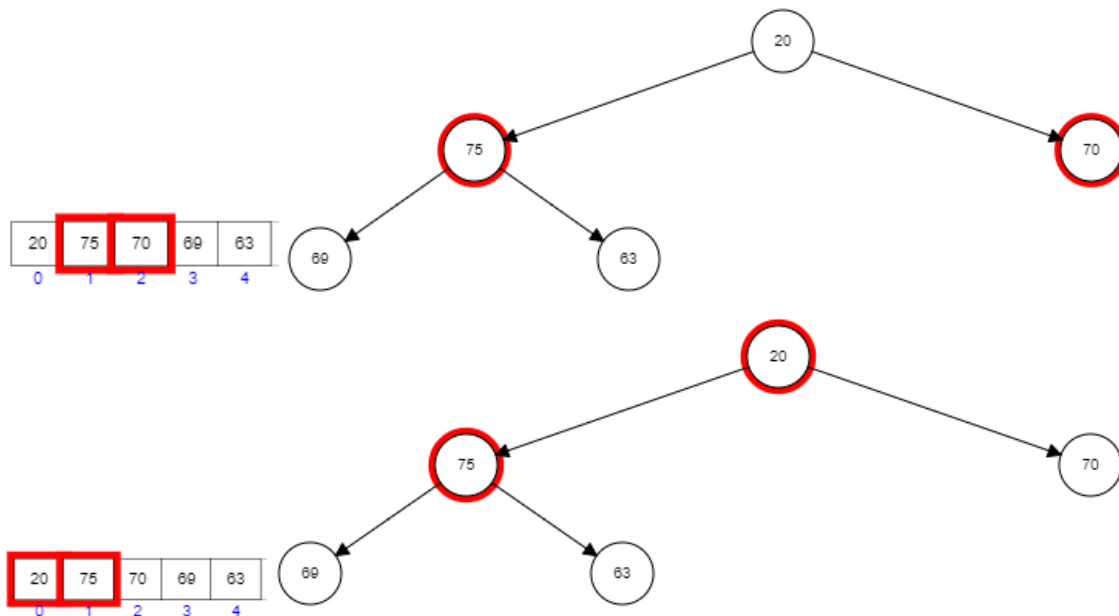
Tutorial 3.1

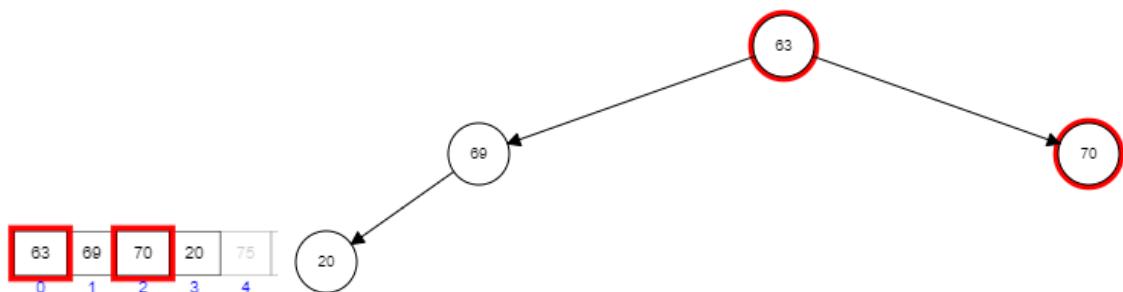
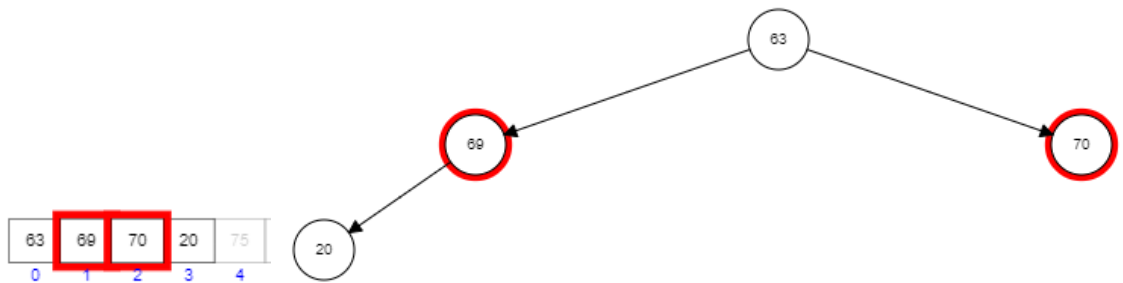
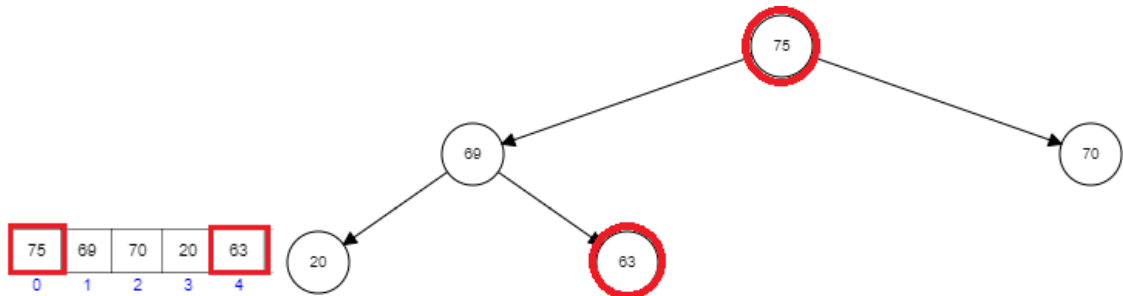
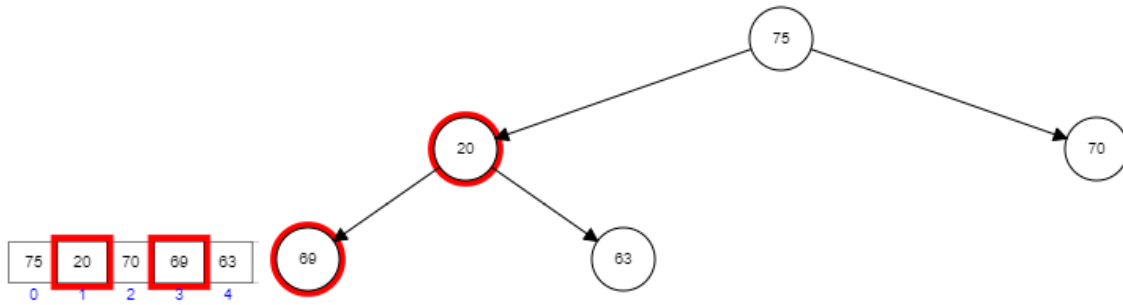
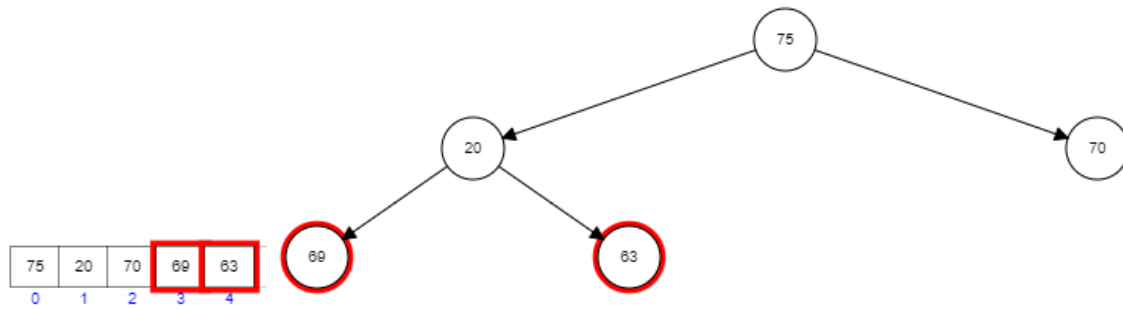
1. Ubahlah function ShellSort diatas sehingga nilai array menjadi terurut secara *descending*.
2. Lalu simpan dengan nama file **Week07_Nama_NIM_3.c**

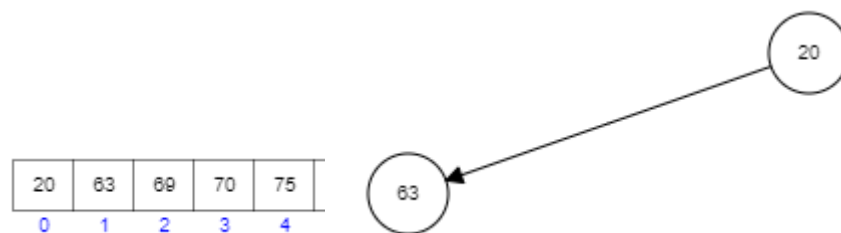
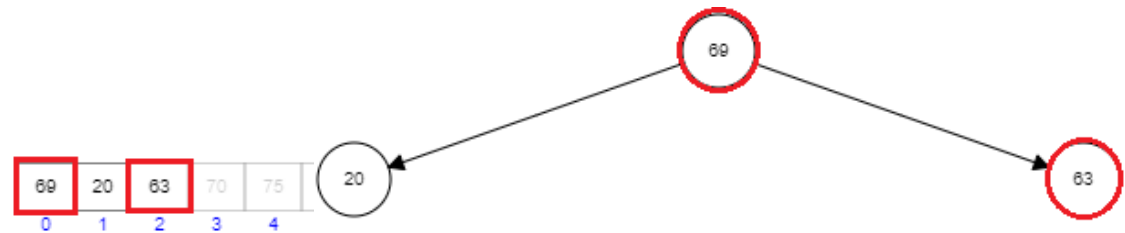
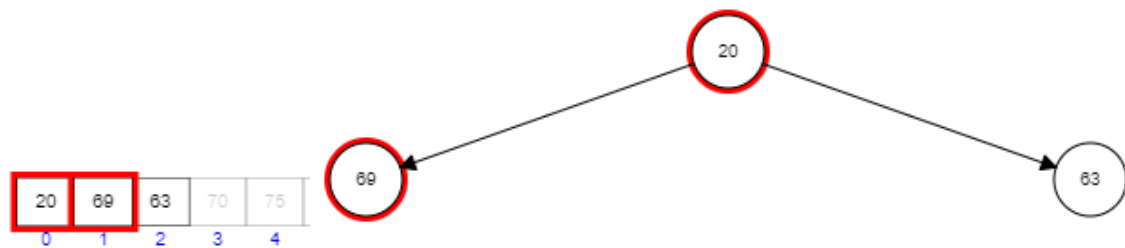
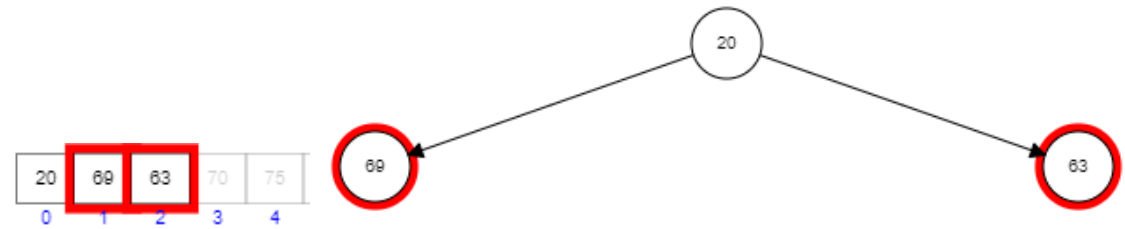
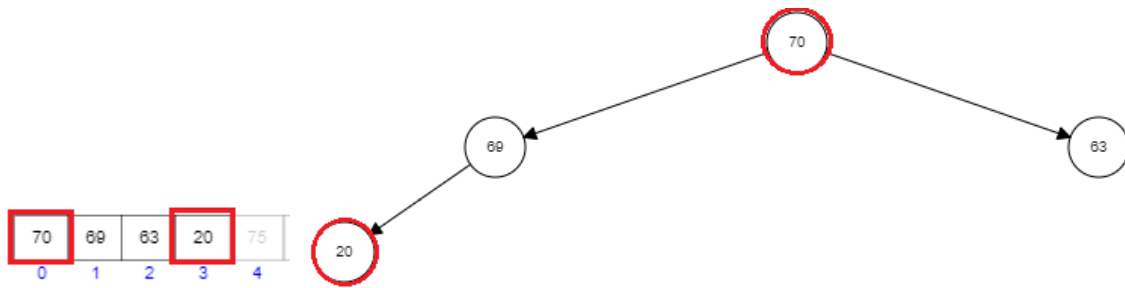
Tutorial 4 – Heap Sort

1. Simulasi

(Sumber: <https://www.cs.usfca.edu/~galles/visualization/HeapSort.html>)







2. Implementasi

```
#include <stdio.h>

void swap(int *bil1, int *bil2){
    int temp;

    temp = *bil1;
    *bil1 = *bil2;
    *bil2 = temp;
}

int heapify(int *bil, int n, int i){
    //int i akan menjadi indeks root sementara-----

    int maks = i;    //Mengambil indeks root sebagai nilai maks-----
    int l = 2*i + 1; //Rumus untuk mengambil indeks array anak di sebelah kiri-----
    int r = 2*i + 2; //Rumus untuk mengambil indeks array anak di sebelah kanan-----

    //Proses pengecekan, apakah nilai dari anak di sebelah kiri > dari maks-----
    if(l < n && bil[l] > bil[maks]){
        maks = l;
    }

    //Proses pengecekan, apakah nilai dari anak di sebelah kanan > dari nilai maks-----
    if(r < n && bil[r] > bil[maks]){
        maks = r;
    }

    //Proses pengecekan, apakah maks masih sama dengan root-----
    if(maks != i){
        swap(&bil[i], &bil[maks]);

        //Melakukan heapify kembali terhadap sub-tree yang terkena dampak dari melakukan swap-----
        heapify(bil, n, maks);
    }
}

int heapSort(int *bil, int n){
    int i;

    //Melakukan heapify dimulai dari parent yang berada pada indeks terbesar-----
    for(i = n/2 - 1; i >= 0; i--){
        heapify(bil, n, i);
    }

    //Menukarkan elemen pada indeks[0] dengan indeks[(n--) - 1]
    //Satu persatu menghilangkan elemen dari heap sehingga tidak lagi terlibat dalam proses heapify.
    //-----
    for(i = n-1; i >= 0; i--){
        //Menukarkan elemen pada indeks[0] ke indeks[(n--) - 1]
        swap(&bil[0], &bil[i]);

        //Melakukan proses max heapify
        heapify(bil, i, 0);
    }
}
```

```
int main(int argc, char const *argv[]){
    int *bil;
    int n;
    int i;

    printf("Jumlah bilangan: ");
    scanf("%d", &n);

    bil = (int *) malloc(sizeof(int) * n);

    for(i = 0; i < n; i++){
        scanf("%d", &bil[i]);
    }

    heapSort(bil, n);

    printf("Hasil pengurutan: ");
    for(i = 0; i < n; i++){
        printf("%d ", bil[i]);
    }
    printf("\n");

    return 0;
}
```

Tutorial 4.1

1. Ubahlah algoritma Heap Sort di atas sehingga nilai array menjadi terurut secara *descending*.
2. Lalu simpan dengan nama file **Week07_Nama_NIM_4.c**

Challenge

1. Buatlah sebuah program data nilai mahasiswa yang disimpan menggunakan *array in double linked list* seperti ini:

```
struct mahasiswa{
    char nama[30];
    int nilaiTugas[15];
    int nilaiAkhir;

    struct mahasiswa *next, *prev;
} mhs;
```

2. Program memiliki menu berikut:

- Tambah Mahasiswa: meminta input user berupa nama mahasiswa dan 14 nilai. Lalu, program akan mengkalkulasi secara otomatis nilai akhir mahasiswa yang didapat dari rata-rata 10 nilai indeks terakhir.
- Sort : Ketika memilih menu ini, muncul pilihan menu Ascending dan Descending, lalu muncul 4 menu lagi (Quick Sort, Merge Sort, Shell Sort, dan Heap Sort). Program akan mengurutkan semua nilai array dari setiap mahasiswa menggunakan algoritma yang dipilih oleh user. Jangan lupa juga untuk meng-update nilai akhir mahasiswa setiap melakukan sort. Lalu program akan menampilkan seluruh data-data mahasiswa yang nilai-nilai nya telah diurutkan.
- Exit : Keluar program.

3. Simpan dengan nama file **week07_NIM_Nama_challenge.c**

REFERENSI